

Pair Programming

Note: This page summarizes James Shore's article on Pair Programming from *The Art of Agile Development*.

Two Minds, One Keyboard, Better Code

Pair programming puts two programmers at one computer. One drives; the other navigates. They switch roles fluidly, constantly communicate, and together accomplish better work more quickly than either could alone.

It sounds strange. It's also extremely powerful—and once you get used to it, a lot of fun.

Why Pair?

The driver types, focusing on tactics: writing clean code that compiles and runs. The navigator focuses on strategy: how the code fits into the overall design, which tests will drive the code forward, which refactorings will improve the codebase.

This division of labor frees the driver to handle the details of rigorous, syntactically correct code without worrying about the big picture. The navigator considers strategic issues without getting lost in implementation. Together, driver and navigator produce higher-quality work more quickly than either could alone.

Research suggests pairing takes about 15% more effort than one person working alone, but produces results faster and with 15% fewer defects. Every team is different, but the trade-off tends to favor pairing.

Beyond productivity, pairing reinforces good habits. Practices like continuous testing and design refinement require discipline. Positive peer pressure helps you actually do the hard things. Knowledge and coding tips spread naturally through the team.

You'll also spend more time in flow—that highly productive state of total focus. Pairing creates a different kind of flow, one that's more resilient to interruptions. Office mates are less likely to interrupt two people working together. When they do, one person handles it while the other keeps their train of thought.

How to Pair

When you start working on a task, ask another programmer to work with you. When someone asks you, make yourself available. Never assign partners—pairs form naturally and shift throughout the day. Over time, pair with everyone on the team. This builds cohesion and spreads knowledge.

When you need a fresh perspective, switch partners. One person stays on the task to bring the new partner up to speed. Even explaining a problem to someone new often helps resolve it.

Switch partners several times per day, even when you don't feel stuck. A natural time to switch is when finishing a task. For big tasks, switch within four hours at most.

Sit side by side, with the monitor clearly visible to both. When driving, place the keyboard directly in front of you—people have a strange tendency to contort themselves rather than moving the equipment.

Produce code through conversation. Think out loud. Talk about your assumptions, short-term goals, general direction, and any relevant history. If you're confused, ask questions. The discussion often enlightens both of you.

Expect to feel tired at the end of the day. Pairs typically feel they've worked harder and accomplished more than when working alone. That's because they have.

Driver and Navigator

When you start pairing, expect to feel clumsy. As driver, you may feel your navigator sees problems faster than you do. She does—navigators have more time to think. The situation reverses when you navigate. It becomes natural with practice.

When navigating, resist the urge to grab the keyboard. Your driver often communicates ideas through both words and code. Give them time to correct their own typos and small mistakes. Use your extra

brainpower for the bigger picture: What other tests do you need? How does this code fit the overall system? Is there duplication to remove? Can the design be clearer?

Help your driver be productive. Think ahead and be ready with suggestions. Keep an index card nearby—rather than interrupting with every thought, jot ideas down and bring them up at a natural break. When questions arise, look up answers while the driver continues working.

Switch roles frequently—at least every half hour, sometimes every few minutes. If you’re navigating and find yourself telling the driver which keys to press, ask for the keyboard. If you’re driving and need a break, hand it off.

Making It Work

Physical Setup

Good pairing stations matter. You need room for two people to sit comfortably side by side. Typical cubicles with workstations in corners don’t work—they force one person behind the other, creating physical and psychological barriers.

Simple six-foot folding tables work well. Get large monitors so both people can see clearly. Some teams plug in two keyboards and mice so each person has their own set.

Comfort and Communication

Pairing is no fun if you’re uncomfortable. Adjust your position and clear debris off the desk. Discuss personal space preferences—some people need more than others.

Communication style takes calibration. New drivers sometimes take over and shut down conversation. To practice, try ping-pong pairing: one person writes a test, the other makes it pass and writes the next test, back and forth.

Transform blunt criticism into collaborative problem-solving. Instead of “This method is too long,” try “Could we make this method shorter?” Adopt an attitude of working together toward better code.

Mismatched Skills

When a senior developer pairs with a junior one, avoid falling into teacher/student mode. Restore peer balance by creating opportunities for both to learn. Ask the junior developer to research something the team needs—give everyone a chance to be an expert.

Common Questions

Isn’t it wasteful to have two people do one person’s work?

Only one keyboard is in use, but there’s more to programming than typing. As Ward Cunningham said, “If you don’t think carefully, you might think that programming is just typing statements in a programming language.” One person programs while the other thinks ahead, anticipates problems, and strategizes.

Do we really have to pair all the time?

That’s a decision for your whole team. Before deciding, try pairing on all production code for a month. You may enjoy it more than expected.

Some tasks are so repetitive they don’t seem to need a pair’s extra brainpower. But before abandoning pairing, ask why your design requires so much repetition. It might signal a design flaw worth fixing.

What if we have an odd number of programmers?

A solo programmer can do productive work that doesn’t involve production code: research technologies, review recent changes with a customer or tester, do exploratory testing, or study the overall design for improvement opportunities.

How can I concentrate with someone talking to me?

If you're having trouble, let your navigator know—they may have suggestions. Often the solution is taking smaller steps. With TDD and tiny increments, you can focus just on the next few lines while your navigator tracks what's left to do.

The Learning Curve

Pairing can feel uncomfortable at first. It requires more collaboration than most programmers are used to. These feelings typically fade after a month or two.

Try it as an experiment: set aside a month where everyone pairs on all production code. Make sure the whole team is on board—the only programmers who try pairing and don't like it are usually those forced into it against their will.

If you're the only one using pairing on your team, you may find teammates break your tests and don't fix them. It's better to get everyone to try it together.

Results

When pairing works well, you experience fewer interruptions and deeper focus. When interrupted, one person handles it while the other maintains context. You slide back into flow immediately. At the end of the day, you feel tired but satisfied.

The team enjoys higher-quality code. Technical debt decreases. Knowledge travels quickly, raising everyone's competence and helping new members integrate faster.

Pairing reduces defects, improves design quality, shares knowledge, supports self-discipline, and reduces distractions—all without sacrificing productivity.

References

- James Shore - Pair Programming — The primary source for this section, from *The Art of Agile Development*.
- Laurie Williams & Robert Kessler, *Pair Programming Illuminated* (2002) — An in-depth exploration of pair programming.
- Cockburn & Williams - The Costs and Benefits of Pair Programming — Research on pairing's productivity and quality impacts.